

Mmap Implementation Reflection
Phillip Miavelstuck and Kabir Jolly

1. Starting with the starter code from Lecture 5, we began by using the [AM335x Technical Reference Manual](#) to find the constant values such as GPIO_BASE_PHYS (Section 2.1 in ARM Cortex-A8 Memory Map), GPIO_OE_OFFSET, and others found in General-Purpose Input/Output (Section 35).
2. After filling in the constants, editing the provided functions to fit our project setup:
 - a. Utilizing the `gpio_map_t` struct, an array of 4 was made to store each of their respective fds, map base, and gpio base values.
 - b. As such, it was no longer needed to pass around a `gpio_map_t* m` since the object was now being accessed through the `gpio_control.c` file directly.
 - i. Furthermore, the `gpio_map_close` function was also edited to access the array directly.
3. To then complete the set function, we are given two constants `GPIO_SETDATAOUT_OFF (offset = 194h)` and `GPIO_CLEARDATAOUT_OFF (offset = 190h)`, which we can utilize to either set the register bit to 0 or 1 for output purposes (LEDs). Then, utilizing the `reg32` provided function, we can get the 32-bit registers and set the corresponding bit to whichever we may need (utilizing bitwise operations as done in the [docs](#) as well). Furthermore, we can define `GPIO_DATAIN_OFFSET (offset = 138h)` to be an input register to then write 0 or 1 for input purposes (button).
4. Following this is the set direction function, where we say whether the pin is going to be an in or out direction pin. Utilizing the constant `GPIO_OE_OFFSET (offset = 134h)`, we can set the value to 0 or 1 as stated by the manual to set the corresponding pin direction correctly (for the LED and buttons).
5. Then, utilizing the `GPIO_DATAIN_OFFSET` constant, we made a function to read through the GPIO Base groups created earlier, and additional bitmasking to show when an input read 1 or 0.
6. After that, utilizing the `GPIO_CLEARDATAOUT_OFF` constant, we created a function to clear the register bit with the same GPIO Base groups and more bitmasking.
7. These implementations were all confirmed through their use of LED and button controls, where the directions and values were accessed and manipulated directly to perform the railroad crossing actions.